

Chapter 1

ASIC Design

1.1 Pipeline Implementation

1.1.1 FETCH UNIT

1.1.2 LOAD-STORE UNIT

1.2 Control and Status Registers

1.3 Exceptions and Interrupts

1.3.1 CLINT Mode Interrupt architecture

1.4 Debug Implementation

1.4.1 Debug Mode Entry

When the debugger requests for the debug entry, the processor enters in the debug mode. Currently we have 3 main debug entry points.

1. ***Haltreq* goes high (Async debug entry):** If the processor is in normal operation (pipeline operation) and it detects the *haltreq* signal, then we also enter the Debug state if the processor is not freezed. As soon as we enter the debug mode, *pending_async_dbg* goes low.
2. *resethaltreq* goes high (Reset debug entry)
3. Single Stepping
4. EBREAK (Sync debug entry)

Debug Mode is a special processor mode used only when a hart is halted for external debugging. Because the hart is halted, there is no forward progress in the normal instruction stream. How Debug Mode is implemented is not specified here.

Following things happen during debug mode entry:

1. Kills all the instructions in the pipeline.
2. All interrupts are masked using *interrupt_dbg_global* signal and also by writing `mstatus[MIE] = 1'b0`
3. All traps are disabled
4. All the debug instructions are executed as they do in M-mode except control instructions (**Branch and Jumps**) irrespective of their destination address, all PC related instructions like **AUIPC** act as illegal instructions.

1.4.2 RESET

If the halt signal (driven by the hart's halt request bit in the Debug Module) or `hasrethaltreq` are asserted when a hart comes out of reset, the hart must enter Debug Mode before executing any instructions, but after performing any initialization that would usually happen before the first instruction is executed.

1.4.3 HALT

Before entering the debug mode, the processor is halted when it encounters any of the debug entry indication through CSR or through external signals. In HALT mode, we do following things:

1. `cause` is updated with the cause of debug
2. `prv` is set to 3 and `v` is set to 0 to indicate Machine Mode.
3. `dpc` is set to the next valid instruction which needs to be executed when we leave the debug mode.
4. Kill all the instructions in the pipeline
5. write `mstatus[MIE] <= 1'b0`, to disable all the Software, timer and external interrupts.
6. Assert *interrupt_dbg_global* signal

1.4.4 DEBUG EXIT SEQUENCE

When we get resumereq_i as 1'b1, following sequence is followe.

1. Fetch PC is retored with the one from dpc.
2. Previllage levels are restores as it was before entering the debug mode.
3. Leave the debug mode.

1.5 ICACHE Design

The processor boots from the address 32'h0000_0000, which is the base address of the instruction memory (BOOT ROM) in our case.

When the first time processor sends the BASE_ADDRESS, we'll have a icache miss. Then the virtual address is given to the TLB block and physical address is given out with a tag_valid signal. In our case, since we have disabled the translation for ease of use, we'll have physical address = virtual address. When the tag_hit is 1'b1 and hit_out is 1'b0, then the state machine inside the **cachefsm** block goes to the service miss state(2'b01).

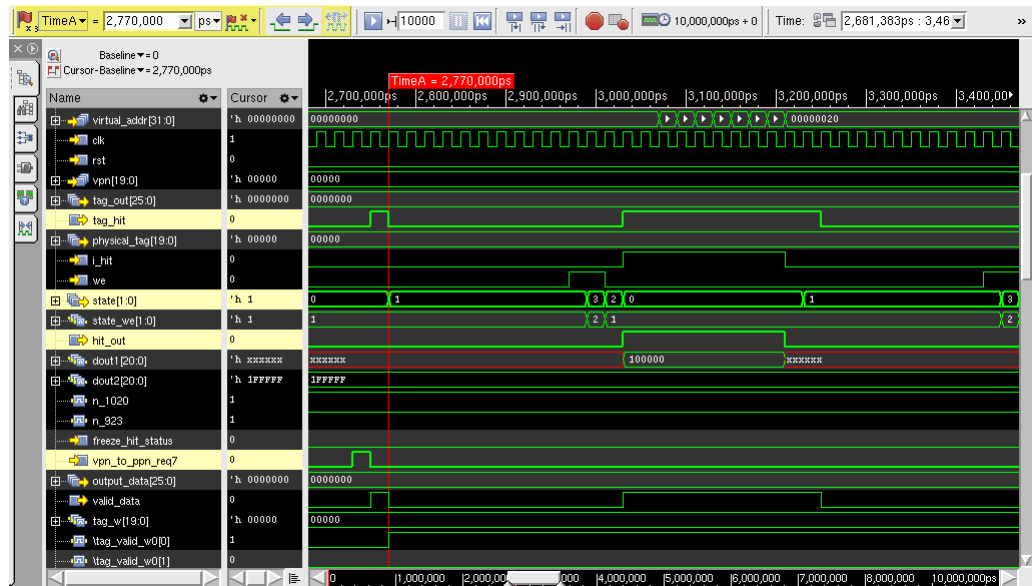


Figure 1.1: Initial icache miss

In the state SERVICE_MISS, the physical address is given to the instruction memory for fetching the instruction in burst. We have a cache line of 256bits, that is 8 words (32-bits).

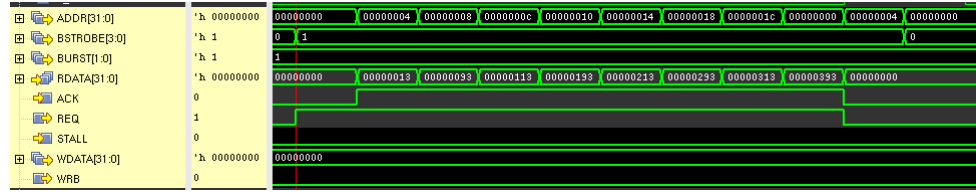


Figure 1.2: Address burst to fetch instruction

When the whole cache line is fetched, the cachefsm signals we to the icache controller for writing the cache line in to the memory. Physical tag along with the tag_valid is also written to the tag array memory. When *we* comes from the cachefsm, the FSM inside the icache controller state *state_we* goes to 2 (2'b10). In state 2, it cache line and tag is written on to the ICACE SRAM, and in next cycle it again comes to state 1.

Parallely, states of cachefsm are also moving. When the state of cachefsm is 2'b10, it raises the vpn_to_ppn_req_3 signal, which goes to the icache controller and the state goes to 0.

This vpn_to_ppn_req_3, is used to read the tag again from the TLB and compare it with the tag coming from the TAG SRAM ARRAY. If it matches, then it is a hit otherwise miss.

When the cache hits, it sends the *instruction[31:0]* to the pipeline unit using block select counter.

Now then the PC goes to 32'h0000_0020, the index increments and we encounter a cache miss, since the tag from the TLB(physical) doesn't match with that from the memory. So signal i_hit goes down. Now the state in cachefsm again goes to SERVICE_MISS. At this moment, the tag_valid[VA] for the next upcoming way is written 1'b1 and cache again fetched the next line from the memory.

1.6 DCACHE Design

1.7 Physical Memory protection

1.8 GLS Issues

1. Hit issue during CACHE miss for the 1st time

CURRENT WAVEFORM

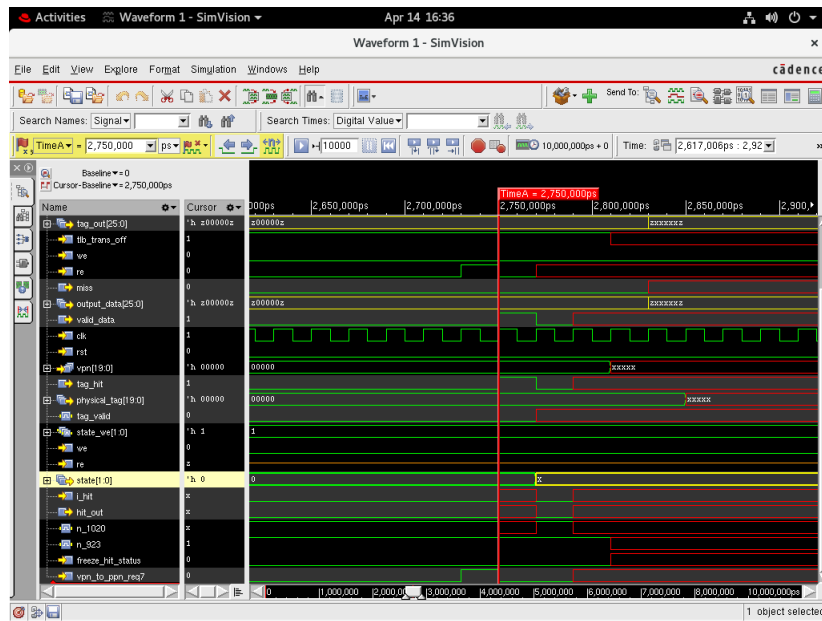


Figure 1.3: Current Waveform

EXPECTED WAVEFORMS

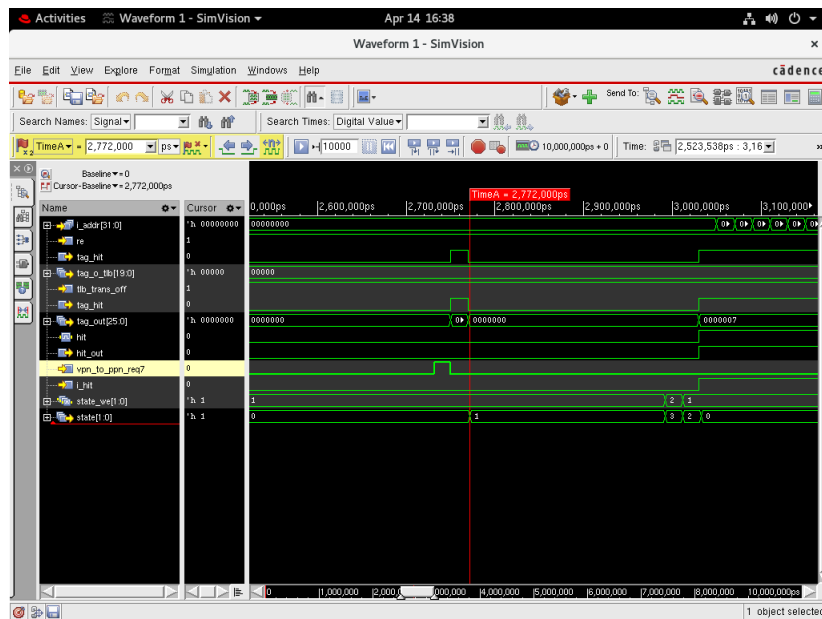
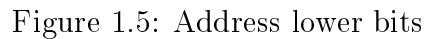


Figure 1.4: Expected Waveforms

For the moment, I have initialised all the Memory output of dout1_mem[20:0]

```
force tb_platform.riscv_platform.cpu1.fdem.mh.icache1.dout1_mem = 21'h1ffff; force
tb_platform.riscv_platform.cpu1.fdem.mh.icache1.dout2_mem = 21'h1ffff; xcelium> force
tb_platform.riscv_platform.cpu1.fdem.mh.icache1.dout1_mem = 21'h1ffff; force tb_platform.riscv
= 21'h1ffff;
```

Then we have the issue at the Address line of the ICACHE (fdem.mh.interface_router.ADDR). Lower 2 bits of the address is going “Z”



3400ns: >> At 3400ns, Release the force on dout1_mem.



SOLUTION: All the memories will be initialised to X in the GLS. Hence the RTL code needs to be designed in such a way that it doesn't read the memory before it writes something on to it. Hence we designed the tag_valid register which holds the value 1'b1, if that cache line is valid.

```
hit = (tag_hit ? ((tag_valid_w1_read && (tag==dout2[(tag_last_bit - tag_start_bit)
      (tag_valid_w0_read && (tag==dout1[(tag_last_bit - tag_start_bit)
      : 1'b0);
```

Figure 1.7: RTL fix

Here when the tag_hit is 1'b1, and tag_valid_w1_read is 1'b0, it doesn't matter what the dout values are and the hit will remain to 1'b0 cleanly.

2. Delay Loops inside the designs: Designs contain many paths with sequential elements, which have no clock to q delay. All the # delays will be removed out from the design once the synthesis is done.

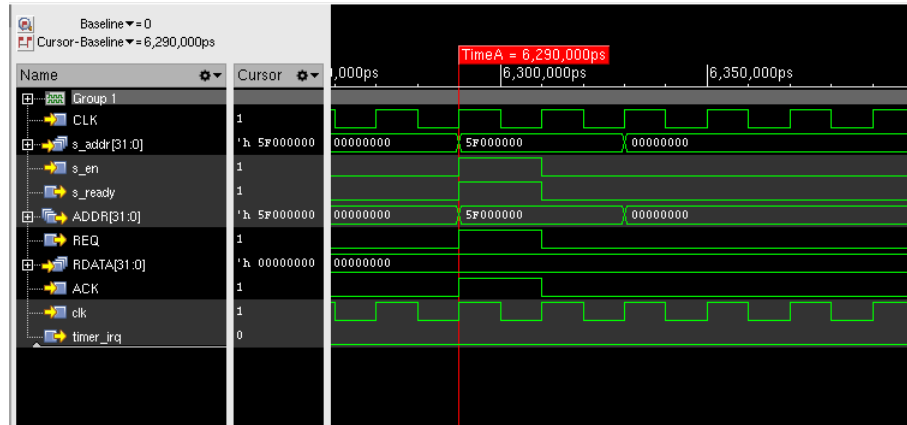


Figure 1.8: CLINT Req-Ack issue

In the above case, *s_en*, comes along with the address *s_addr*. The *s_ready* has to be 1'b1 in the next clock cycle, but in functional it works fine since we have given # delay to the flops, but doesn't work in zero delay or unit delay simulations.

```
// READY assertion
always @(posedge CLK or posedge RST) begin
    if(RST) s_ready <= 1'b0;
    else begin
        if(s_en && ~s_ready) s_ready <= 1'b1;
        else s_ready <= 1'b0;
    end
end
```

Figure 1.9: RTL code

SOLUTION: add the ***-seq_udp_delay 200ps*** to the script to make the output of all the flops to come after a delay of 200ps to model the tcq for correct functionality.